23CSE455 DESIGN PATTERNS

Design Pattern Analysis in a Spring Cloud-Based Hospital Management System

CASE STUDY REPORT - DRAFT

Team	[PENDING exact assigned team name]
Application	Spring Cloud Hospital Management System
Faculty approval	[PENDING approval evidence]
Report date	2026-10-03
Submission repository	23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
[PENDING]	[PENDING]	[PENDING]

Abstract

This study examines a Spring Cloud hospital-management application at the supplied baseline, evolves it through three approved service changes, and compares two design patterns across four languages. The original archive contains Eureka, a Gateway, Auth, Patient and Appointment modules. Its confirmed baseline inventory is 11 collaborations across six pattern labels. The evolved application adds Doctor, Pharmacy, Laboratory, Billing and Notification. The full Maven reactor passed 17 unit tests and an all-service Gateway smoke flow passed 14 of 14 expected HTTP outcomes. Strategy and Adapter demonstrations in Java, C++, Python and JavaScript passed equivalent checks. Team identity, student contributions, upstream source URL/commit, faculty approval and a verbatim desktop model transcript were not supplied.

Contents

1 Application and architecture

2 Part 1 pattern discovery

3 Part 2 evolution and refactoring

4 Part 3 cross-language comparison

5 LLM records and human verification

6 Findings, contributions and references

7 Team folder and datasets

Report status: technical work and evidence are included. Replace highlighted pending fields with assigned team/student details and approved sign-off before submission.

1 Application and architecture

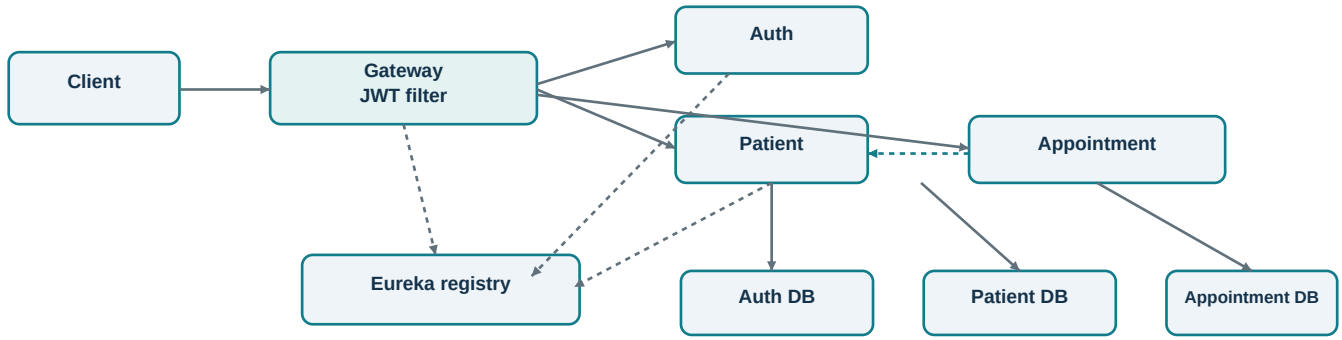
1.1 Original source and environment

The baseline is the supplied archive Hospital-Management-System-Using-MicroServices-And-SpringBoot-main.zip, SHA-256 fc355ebd1b3c5b4f94fb3dd39a71737ebf4f16610da10a5ae9ae29f71e4fc962. No upstream repository URL, tag or exact commit could be recovered. The archive is kept outside the report folder because it contains configuration secrets. The baseline has five Spring Boot modules: Eureka Server, Hospital Gateway, Auth, Patient and Appointment.

The evolved build uses Java 17, Spring Boot 4.1.0, Spring Cloud 2025.1.2, Maven 3.9.16 and PostgreSQL 17.11 for the isolated verification run. The ten evolved Maven modules add Doctor, Pharmacy, Laboratory, Billing and Notification. Runtime secrets and database settings are supplied through environment variables.

1.2 Architecture and scope

Clients enter through the JWT-filtered Gateway. Gateway routes use Eureka service names. Appointment makes load-balanced RestTemplate and WebClient calls for patient validation. Auth, Patient and Appointment use Spring Data JPA repositories and PostgreSQL. Spring Cloud Netflix provides Eureka integration; Gateway supports route filters and cross-cutting concerns [2, 3].

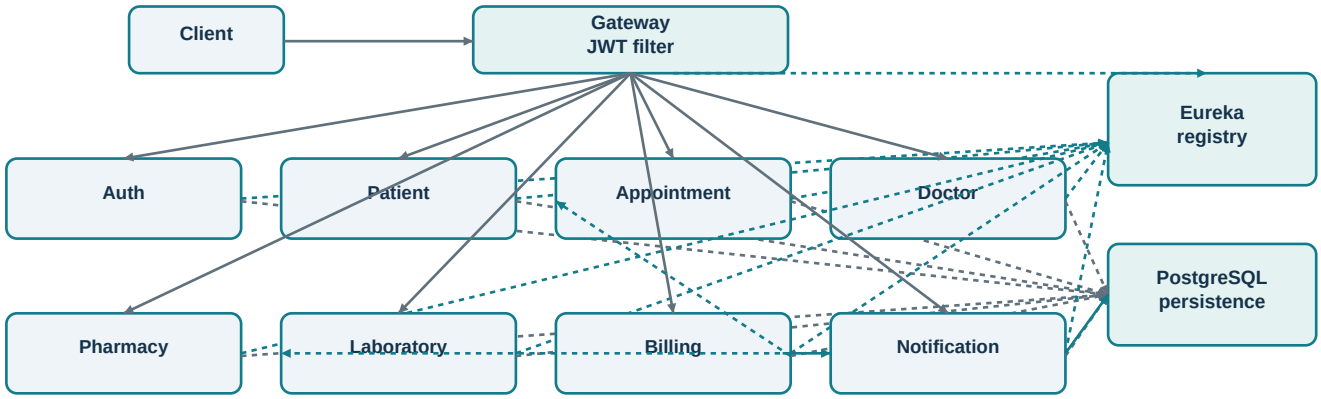


Solid: request/data flow Dashed: service discovery or validation

Figure 1. Baseline service request and discovery relationships. Source: inspected archive configuration and application classes.

1.3 Evolved architecture

Part 2 added Doctor, Pharmacy, Laboratory, Billing and Notification behind the Gateway and registered those modules with Eureka. Appointment validates both patient and doctor; appointment and billing workflows send bounded best-effort notifications. Billing can use the deterministic local provider or the configured external HTTP adapter.



Gateway routes to all domain services; Eureka discovery is dashed. Appointment validates Patient/Doctor and calls Notification; Billing calls payment and Notification.

Figure 2. Evolved services and principal interactions. Each service owns its persistence mappings; verification used one disposable PostgreSQL database.

1.4 Scope and run instructions

From Part2/code/evolved/, set JWT_SECRET (at least 32 bytes) and DB_PASSWORD if needed, then run mvn test. Verification used the local simulated provider and sent no real payment. Full unit and smoke summaries are under data/test_result/part2-unit-tests.md and data/test_result/part2-smoke.md; focused change results are in each Change folder. The external-payments profile also requires SPRING_PROFILES_ACTIVE and PAYMENTS_API_URL.

2 Part 1 pattern discovery

2.1 Method and baseline totals

Candidates were identified from declarations, annotations, route configuration and call sites. Each confirmed pattern requires participant and interaction evidence. Framework repository proxies are recorded as framework implementations. The inventory and count files are Part1/patterns.csv and Part1/Pattern_count.csv.

Measure	Baseline result
Confirmed pattern types	6
Confirmed pattern instances	11
Distinct concrete participant classes	16 (14 application classes plus named Spring RestTemplate and ReactiveLoadBalancerClientFilter)
Unique mapped class/interface/function symbols	27; shared symbols counted once
Rejected / uncertain	7 rejected; 1 uncertain
Other excluded/unverified candidates	Builder, absent DTO, configured tracing without runtime verification

2.2 Confirmed pattern inventory

Pattern type	Count	Representative collaboration(s)
API Gateway	1	Hospital API edge routing
Service Registry and Discovery	1	Eureka service registration and lookup
Client-side Load Balancing	3	Gateway route lookup; Appointment RestTemplate; Appointment WebClient
Repository	3	User, Patient and Appointment persistence
Intercepting Filter	1	JWT request gate at Gateway
Remote Procedure Invocation	2	Appointment patient validation using both clients

Baseline evidence points to the sanitized source copy in data/code/original/. Part1/patterns.csv contains the detailed participant roles, source line ranges and intent evidence.

2.3 Rejected, uncertain and absent candidates

Seven candidates were rejected in the baseline: Strategy, Observer/event-driven messaging, Service Layer, Singleton, Factory, Circuit Breaker/fallback and Config Server. Full MVC was uncertain because the REST controllers and entities have no View component. Builder use in dependencies is excluded as framework use; DTO is absent from the original registration response; tracing is configured but was not runtime-verified end to end. Doctor, Pharmacy, Laboratory, Billing and Notification were absent in the supplied archive and are treated as Part 2 additions.

3 Part 2 evolution and refactoring

3.1 Change acceptance and pattern effects

Change	Implementation and pattern effects	Verification result
Change 1	Doctor CRUD and appointment doctor validation; Pharmacy medication orders; Laboratory test orders. Added repositories, client-side balancing and RPI.	Doctor route and both appointment client variants passed; unknown Doctor returned 404; Pharmacy/Lab routes passed.
Change 2	Billing invoices/payments with PaymentProvider Strategy, HttpPaymentProvider Adapter, custom Circuit Breaker and BillingService boundary.	Local payment marked invoice PAID; HTTP stub success/503 cases and breaker transitions passed; failure stored PAYMENT_RETRY_REQUIRED.
Change 3	Notification persistence and best-effort appointment/payment workflow hooks.	Four records persisted in smoke run; listing passed; sender failure was retained as DELIVERY_FAILED without undoing the primary operation.

3.2 Refactoring and behavior

Registration now returns a safe response DTO instead of the User entity. Appointment validation distinguishes malformed input (400), missing patient/doctor (404) and unavailable downstream service (503). JWT signing keys and database credentials come from environment variables. Unit tests cover the safe response and the error mapping.

3.3 Before/after source metrics

Java source LOC counts nonblank production Java lines and excludes blank lines, standalone line/block comments, tests, generated code and build output. Baseline: 19 Java files and 419 LOC across the five original modules. The same five evolved modules: 27 files and 554 LOC. The five added services contribute 31 Java files and 337 LOC; evolved total: 58 files and 891 LOC. LOC growth describes scope and does not prove a design

Intermediate source snapshots after Changes 1 and 2 are unavailable, so stage-wise LOC deltas are omitted.
Per-version counts are in Part2/pattern_evaluation.csv; missing LOC is not estimated.

3.4 Test results and pattern counts

Evidence	Result
Full Maven reactor	17 unit tests; 0 failures; 0 errors across 10 modules
Gateway smoke flow	14/14 expected HTTP outcomes passed across all services
Original code	6 types; 11 instances
After Change 1	6 types; 18 instances (+7)
After Change 2	10 types; 24 instances (+6)
After Change 3	10 types; 30 instances (+6)

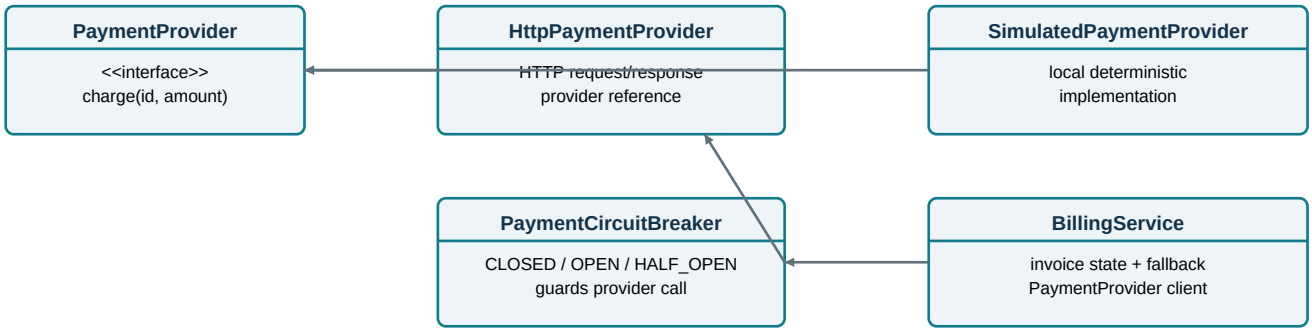


Figure 3. Billing collaborators for provider selection, adaptation and resilience. See Part2/diagrams/evolved_class_diagram.mmd for the editable class view.

The new modules extend service ownership and add validation, payment and notification paths. Strategy and Adapter isolate provider selection and protocol conversion. The Circuit Breaker and best-effort notifications limit selected failure propagation. The demonstrated remote calls remain synchronous; notification uses bounded HTTP rather than a broker. Local notification only logs and records; external payment was tested with a local HTTP stub, not a real provider.

4 Part 3 cross-language comparison

Strategy and Adapter are selected because both are evidenced in Billing: PaymentProvider has simulated and HTTP implementations, while HttpPaymentProvider converts an external HTTP protocol into the application contract. The examples use the same inputs and expected outputs in Java, C++, Python and JavaScript.

4.1 Common specifications

Strategy: 10,000 cents produces standard and premium totals of 9,000 and 8,000 cents. Adapter: 2,599 cents enters as integer cents; the legacy gateway accepts decimal dollars; the adapter returns APPROVED, 2,599 cents and reference legacy-25.99. Each language has separate production code and test runner.

4.2 Static/dynamic measures and test results

Pattern	Language	LOC	Max CC	Test result	Median process ms
Strategy	Java	32	1	2/2 expected value checks passed	114.722
Strategy	C++	26	1	2/2 expected value checks passed	14.604
Strategy	Python	17	1	1/1 test passed; both expected values matched	25.909
Strategy	JavaScript	23	1	2/2 expected value checks passed	48.374
Adapter	Java	31	1	1/1 normalized receipt passed	78.031
Adapter	C++	43	1	1/1 normalized receipt passed	14.421
Adapter	Python	21	1	1/1 test passed; normalized receipt matched	48.151
Adapter	JavaScript	27	1	1/1 normalized receipt passed	48.954

All eight test runners passed. Java and C++ were compiled before timing. Each snippet had one warm-up and five measured process launches on the same machine; medians are reported. Time includes process/runtime startup, execution and stdout capture; excludes compilation. No stdin is read. Raw runs are retained in Part3/evidence/runtime_runs.md.

Maximum function cyclomatic complexity was manually counted with one common McCabe rule: 1 plus each if/else-if, loop, case, catch, ternary, && or || branch. All production functions in these intentionally simple examples have complexity 1; top-level entry guards are not functions. LOC is nonblank production lines excluding comments/tests. The examples contain no mutable shared state, so concurrency behavior is not measured. Java/C++ use explicit contracts; Python/JavaScript use structural object shapes. Python Protocol can express structural subtyping for static checkers [6]; the Java Language Specification defines interfaces as implementable common supertypes [5]; C++ virtual functions provide dynamic binding [7]; ECMAScript does not require a class-based object model [8].

These micro-examples omit I/O complexity and the timing values are dominated by process startup. They should not be interpreted as algorithm speed comparisons. Versions: OpenJDK 17.0.20.1; MinGW GCC 6.3.0-1 (C++17); Python 3.14.4; Node.js 24.15.0.

4.3 Technology references

Spring Cloud Netflix documents Eureka registration/discovery and LoadBalancer integration [2]. Spring Cloud Gateway documents its route/global-filter chain [3]. These references explain framework behavior; the pattern inventory is still grounded in local source interactions.

5 LLM records and human verification

The Part 3 prompt is in Part3/llm/part3_prompt.txt. Inputs are Billing PaymentProvider, HttpPaymentProvider and SimulatedPaymentProvider. Generated code is under Part3/Strategy and Part3/Adapter; output details are in Part3/llm/cross_language_output.md. The audit row in Part3/llm/llm.csv is marked partial because no verbatim transcript export was available. Verification used equivalent tests; no independent second-model or blinded experiment is claimed.

6 Findings, contributions and references

6.1 Findings

The baseline evidence is strongest at the distributed architecture level: Eureka discovery, Gateway routing/load balancing, an intercepting JWT filter, repositories and synchronous remote calls. Part 2 adds five domain services and introduces Strategy, Adapter, Circuit Breaker and Service Layer instances. Tests confirm documented positive and failure outcomes. Limitations include synchronous inter-service calls, no real notification transport, no real payment integration, unavailable intermediate code snapshots, missing upstream commit metadata and no submitted repository screenshot.

6.2 Contributions

Roll number	Student	Batch	Actual contribution
[PENDING]	[PENDING]	[PENDING]	Team members must enter their actual work and evidence.

6.3 References

[1] Amrita Vishwa Vidyapeetham, 23CSE455 Design Patterns Case Study Instructions and Case Study Report template, supplied course PDFs.

[2] Spring, Spring Cloud Netflix Reference: https://docs.spring.io/spring-cloud-netflix/reference/.

[3] Spring, Spring Cloud Gateway Reference: https://docs.spring.io/spring-cloud-gateway/reference/.

[4] Spring, Spring Data JPA Reference: https://docs.spring.io/spring-data/jpa/reference/.

[5] Oracle, The Java Language Specification, Java SE 17, Chapter 9: https://docs.oracle.com/javase/specs/jls/se17/jls17.pdf.

[6] Python Software Foundation, typing - Support for type hints: https://docs.python.org/3/library/typing.html.

[7] ISO C++ working draft, Virtual functions: https://eel.is/c++draft/class.virtual.

[8] Ecma International, ECMAScript Language Specification: https://tc39.es/ecma262/.

[9] JJWT project README: https://github.com/jwtk/jjwt/blob/main/README.adoc.

7 Team folder and datasets

The A1 folder follows the supplied tree. Baseline source and test evidence are under data/code/ and data/test_result/. Part 1 inventory and counts are in Part1/; Part 2 evolution and counts are in Part2/; Part 3 metrics and the available audit record are in Part3/llm/. The team ID, student details and actual contributions were not supplied. Faculty approval and the required team-folder upload screenshot remain pending.

Submission completion checklist

Item	Status
Technical source and Part 1-3 evidence	Prepared in this folder
Team ID and students	Pending
Faculty approval evidence	Pending
Verbatim earlier model transcript	Unavailable in task export
GitHub team-folder upload screenshot	Pending upload